

Medicare — Hospital Appointment Management System with AI Health Assistant

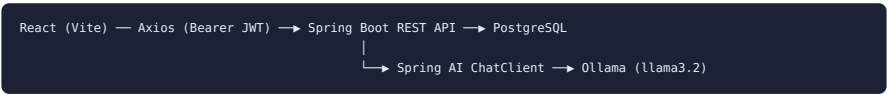
1. Overview

Medicare is a full-stack hospital appointment booking system with three user roles — **Patient**, **Doctor**, and **Admin** — plus an integrated AI health assistant that helps patients describe symptoms, get pointed toward the right specialist, and find doctors in the system.

Layer	Stack
Backend	Java 21, Spring Boot (Spring Boot Parent 4.1.0), Spring Data JPA / Hibernate, Spring Security, Spring AI (Ollama starter), Maven
Database	PostgreSQL (<code>medicare_db</code>)
Auth	JWT (jjwt 0.12.6), BCrypt password hashing
AI	Spring AI <code>ChatClient</code> over a local Ollama server running <code>llama3.2</code>
Frontend	React (Vite), React Router, Axios

Backend runs on the default Spring Boot port (8080); frontend (Vite dev server) runs on `http://localhost:5173`, which is the only origin allowed by CORS.

2. Architecture



The backend follows a layered **Controller** → **Service** → **Repository** pattern:

- **Controllers** — thin, handle HTTP mapping, delegate to services.
- **Services** — business logic, validation rules, orchestration (e.g. appointment status transitions, notification creation).
- **Repositories** — Spring Data JPA interfaces over the entities.
- **DTOs** — request/response objects for every endpoint (18 DTOs), keeping entities out of the wire format.
- **Security** — a custom `JwtAuthenticationFilter` runs before Spring Security’s username/password filter, resolving the user and role from the token on every request.
- **GlobalExceptionHandler** — a single `@RestControllerAdvice` catches `RuntimeException` and returns `400 Bad Request` with `{ "error": "<message>" }`.

3. Data Model

Entity	Table	Key fields	Relationships
User	users	id, name, email, password (hashed), phone, role (enum: <code>PATIENT</code> , <code>DOCTOR</code> , <code>ADMIN</code>), appointmentReminders, emailNotifications	—
Doctor	doctors	id, name, email (unique), phone, specialization, qualification, experience, available	@ManyToOne → Department
Department	departments	id, name (unique)	—

Appointment	appointments	id, appointmentDate, appointmentTime, reason, status (enum: BOOKED , CONFIRMED , COMPLETED , CANCELLED)	@ManyToOne → User (patient), @ManyToOne → Doctor
Notification	notifications	id, title, message, type, read, createdAt	@ManyToOne → User

Note: `Doctor` is a separate entity from `User` — a doctor's login and profile live in `Doctor`, not in the `User / Role.DOCTOR` table, while patients and admins are `User` records.

4. Authentication & Authorization

- **Registration** (`POST /api/auth/register`) — creates a `User` with a BCrypt-hashed password and a role. Validated fields: name, email (must be a valid email), password (min 6 chars), phone, role.
- **Login** (`POST /api/auth/login`) — verifies the password with `BCryptPasswordEncoder.matches()` and issues a JWT signed with HMAC-SHA, subject = email, **1-hour expiry**.
- **JWT filter** — reads the `Authorization: Bearer <token>` header on every request, extracts the email, loads the `User`, and sets a Spring Security authority of `ROLE_<role>` (e.g. `ROLE_ADMIN`) in the security context. No refresh-token mechanism exists; the frontend simply asks the user to log in again after expiry.
- **Password changes** (`PUT /api/users/change-password`) — requires the current password to match before setting the new (re-hashed) one.

Route-level authorization (`SecurityConfig`)

Path pattern	Access
<code>/api/auth/**</code>	Public
<code>/api/ai/**</code>	Any authenticated user
<code>/api/admin/**</code>	ADMIN
<code>/api/users/patients</code> , <code>/api/users</code> (list)	ADMIN
<code>POST/PUT/DELETE /api/doctors/**</code>	ADMIN
<code>PUT /api/doctors/me</code>	DOCTOR
<code>GET /api/doctors/**</code>	Any authenticated user
<code>/api/doctor/**</code> (doctor appointment endpoints)	DOCTOR
<code>GET /api/notifications/admin</code>	ADMIN
<code>/api/notifications/**</code> (other)	Any authenticated user
<code>GET /api/appointments/admin</code>	ADMIN
<code>/api/appointments/**</code> (other)	PATIENT
Everything else	Any authenticated user

CSRF protection is disabled (stateless JWT API). CORS allows only `http://localhost:5173`, methods `GET/POST/PUT/DELETE/OPTIONS`, all headers, credentials enabled.

5. REST API Reference

Base URL: `http://localhost:8080/api`

Auth (`/auth`) — public

Method	Path	Body	Notes
POST	<code>/auth/register</code>	<code>RegisterRequest</code> (name, email, password, phone, role)	Returns <code>UserResponse</code> , 201
POST	<code>/auth/login</code>	<code>LoginRequest</code> (email, password)	Returns <code>LoginResponse</code> (token, email, role)

Users (`/users`)

Method	Path	Access	Notes
GET	<code>/users/me</code>	self	Current profile
PUT	<code>/users/me</code>	self	Update profile
PUT	<code>/users/notification-preferences</code>	self	Toggle appointment reminders / email

PUT	/users/change-password	self	notifications Requires current password
GET	/users/patient/{patientId}	doctor/admin	Fetch a specific patient (validates target is a PATIENT)
GET	/users/patients	ADMIN	All patients
GET	/users	ADMIN	All users

Doctors (/doctors)

Method	Path	Access	Notes
POST	/doctors	ADMIN	Create doctor account (hashes password, checks unique email)
GET	/doctors	authenticated	List all
GET	/doctors/search?name=	authenticated	Case-insensitive name search
GET	/doctors/specialization/{specialization}	authenticated	Filter by specialization
GET	/doctors/department/{departmentId}	authenticated	Filter by department
GET	/doctors/available	authenticated	Doctors with available = true
GET	/doctors/me	DOCTOR	Own profile
PUT	/doctors/me	DOCTOR	Update own profile
GET	/doctors/{id}	authenticated	Doctor by id
PUT	/doctors/{id}	ADMIN	Update any doctor
DELETE	/doctors/{id}	ADMIN	Delete doctor
GET	/doctors/dashboard	DOCTOR	Appointment counts by status
GET	/doctors/appointments	DOCTOR	All own appointments
GET	/doctors/appointments/today	DOCTOR	Today's appointments
PUT	/doctors/appointments/{id}/confirm	DOCTOR	BOOKED → CONFIRMED
PUT	/doctors/appointments/{id}/complete	DOCTOR	CONFIRMED → COMPLETED

(DoctorAppointmentController under /api/doctor/appointments duplicates equivalent confirm/complete/cancel/today/detail endpoints for the doctor’s own appointments.)

Departments (/departments)

Method	Path	Access
POST	/departments	authenticated (no explicit role restriction beyond default)
GET	/departments	authenticated
GET	/departments/{id}	authenticated
DELETE	/departments/{id}	authenticated

Appointments — patient side (/appointments , requires PATIENT)

Method	Path	Notes
POST	/appointments	Book; validates doctor exists, date is present/future, and rejects a double-booked doctor/date/time slot
GET	/appointments/my	All of the caller’s appointments
GET	/appointments/my/upcoming	Future, non-cancelled
GET	/appointments/my/history	Past / completed
PUT	/appointments/{id}/cancel	Blocked if already CANCELLED or COMPLETED ; ownership checked
GET	/appointments/{id}	Single appointment (ownership checked)
GET	/appointments/admin	ADMIN only — all appointments

Notifications (/notifications)

Method	Path	Access
GET	/notifications	self — all notifications, newest first
GET	/notifications/unread	self
GET	/notifications/unread/count	self
GET	/notifications/admin	ADMIN — all notifications
PUT	/notifications/{id}/read	self
PUT	/notifications/read-all	self

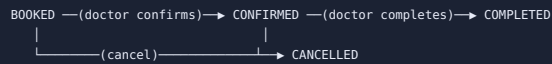
Admin (/admin)

Method	Path	Notes
GET	/admin/dashboard	System-wide metrics (doctors, patients, appointments by status, etc.)

AI Assistant (/ai)

Method	Path	Body	Access
POST	/ai/chat	AIChatRequest (message, optional conversation history)	Any authenticated user

6. Appointment Lifecycle



Rules enforced in `AppointmentService` : - A doctor cannot be double-booked for the same date and time (`existsByDoctorIdAndAppointmentDateAndAppointmentTime`). - Only `BOOKED` appointments can be confirmed; only `CONFIRMED` appointments can be completed. - An appointment already `CANCELLED` or `COMPLETED` cannot be cancelled again. - Every confirm / complete / cancel action creates a `Notification` for the relevant user (`APPOINTMENT_CONFIRMED`, `APPOINTMENT_COMPLETED`, etc.). - The doctor dashboard and admin dashboard both derive their counts from `countByDoctorId` / `countByDoctorIdAndStatus` queries rather than loading full lists.

7. AI Health Assistant

Implemented with Spring AI's `ChatClient` against a local Ollama instance (`llama3.2`, temperature 0.7, `http://localhost:11434`). Each request rebuilds a system prompt plus the recent conversation history and sends it as a single call — the service is stateless between requests; the frontend resends prior turns as `conversation` in the request body.

Safety rules baked into the system prompt: no diagnosis, no medication or dosage advice, no certainty about symptom causes, and a direction to seek immediate care for emergency or life-threatening symptoms.

Intent detection. The model is instructed to classify each message into one of: - `SPECIALIST_SEARCH` — the user asks which *type* of doctor to see (e.g. “which specialist should I see for headaches?”). The model responds with a `SPECIALIST: <name>` line. - `DOCTOR_SEARCH` — the user wants to see actual doctors registered in Medicare (e.g. “show me neurologists”). - `APPOINTMENT_REQUEST` — the user wants to book/schedule an appointment.

The backend parses the model's structured output (intent + specialist name) and, for specialist/doctor searches, queries `DoctorRepository` for matching, available doctors, returning them in `AIChatResponse.doctors` alongside the natural-language reply. The prompt explicitly tells the model the specialist suggestion is general guidance, not a diagnosis, and that booking itself happens through the app's normal flow rather than through the AI response.

8. Frontend

React app built with Vite, using React Router for role-scoped routing and a custom `ProtectedRoute` component that gates a route by `allowedRoles`.

- `AuthContext` stores `token` and a normalized `user` object (`{ email, role }`) in `localStorage` and exposes `login`, `logout`, and `isAuthenticated`.
- **Axios instance** (`api/axios.js`) sets `baseUrl`: `http://localhost:8080/api`, attaches `Authorization: Bearer`

<token> to every request except `/auth/login` and `/auth/register`, and logs each request/response to the console.

- **Login** posts to `/auth/login` and redirects based on the returned role: `ADMIN` → `/admin/dashboard`, `DOCTOR` → `/doctor/dashboard`, `PATIENT` → `/patient/dashboard`.

Route map (role-gated via `ProtectedRoute`)

Role	Routes
Patient	<code>/patient/dashboard</code> , <code>/patient/ai-assistant</code> , <code>/patient/doctors</code> , <code>/patient/book-appointment</code> , <code>/patient/appointments</code> , <code>/patient/appointments/:id</code> , <code>/patient/upcoming</code> , <code>/patient/history</code> , <code>/patient/profile</code> , <code>/patient/settings</code>
Doctor	<code>/doctor/dashboard</code> , <code>/doctor/profile</code> , <code>/doctor/change-password</code> , <code>/doctor/patients</code> , <code>/doctor/patients/:patientId</code> , <code>/doctor/appointments</code> , <code>/doctor/appointments/today</code> , <code>/doctor/appointments/:id</code>
Admin	<code>/admin/dashboard</code> , <code>/admin/doctors</code> , <code>/admin/doctors/new</code> , <code>/admin/doctors/:id</code> , <code>/admin/doctors/:id/edit</code> , <code>/admin/patients</code> , <code>/admin/patients/:id</code> , <code>/admin/departments</code> , <code>/admin/notifications</code>
Any authenticated	<code>/notifications</code>
Public	<code>/login</code> , <code>/unauthorized</code> , <code>*</code> (404)

9. Setup & Running Locally

Prerequisites: Java 21, Maven, Node.js, PostgreSQL, and Ollama with the `llama3.2` model pulled (`ollama pull llama3.2`).

1. **Database** — create a PostgreSQL database named `medicare_db`. Hibernate is set to `ddl-auto=update`, so tables are created/updated automatically on startup.
2. **Backend config** — in `medicare-backend/src/main/resources/application.properties`, set `spring.datasource.username` / `spring.datasource.password` to your PostgreSQL credentials.
3. **Start Ollama** — `ollama serve` (default `http://localhost:11434`), with the `llama3.2` model available.
4. **Run backend** — from `medicare-backend/`: `mvn spring-boot:run`. Starts on port `8080`.
5. **Run frontend** — from `medicare-frontend/`: `npm install && npm run dev`. Starts on port `5173` (Vite default).
6. **Register a user** via `POST /api/auth/register`, or seed a `Department` and `Doctor` first (doctor accounts are created by an admin through `POST /api/doctors`).

10. Known Limitations / Suggested Improvements

- **Hardcoded JWT secret** (`JwtService`) — should be moved to an environment variable / config property, and rotated, especially if the repo is public.
- **Database credentials in `application.properties`** — same recommendation; use environment variables or a secrets manager, and avoid committing real credentials to version control.
- **No token refresh** — the 1-hour JWT simply expires; there's no refresh-token endpoint, so users must log in again.
- **Verbose console logging** in `JwtAuthenticationFilter` and the Axios interceptor (method, URL, token presence, decoded role) — fine for development, but should be removed or put behind a debug flag before any production deployment.
- **Overlapping doctor-appointment endpoints** — `AppointmentController`'s admin path and `DoctorController`'s

appointment methods duplicate functionality also found in `DoctorAppointmentController`; consolidating these would simplify the controller layer.

- `GlobalExceptionHandler` catches only `RuntimeException` and always returns `400`; distinguishing not-found vs. validation vs. conflict errors (404/400/409) would make the API easier for a frontend or third party to consume correctly.